

Slot 8 · Labels and class imbalance

File: `imbalance.py` (and `labels_<yourname>.csv`) · **Done when:** `uv run pytest tests/test_slot08.py` shows 3 passed · **Time:** full slot

Today's idea

Three parts. (1) Even careful humans disagree on labels. (2) When one class is very rare, **accuracy lies**. (3) A contest to catch as many bad deliveries as possible.

Part 1 · Label 20 orders, then compare with a neighbour (15 min)

Make your own labelling sheet. Everyone gets the **same 20 orders**:

```
uv run python imbalance.py --make-labels yourname
```

Open `labels_<yourname>.csv` in VS Code or any text editor (not Excel — it can change the file). For each order, was the delivery **acceptable**? At the end of each line, replace the `""` with **1** (acceptable) or **0** (not acceptable). Decide on your own. No discussion.

Share your file with your neighbour (AirDrop is fine). Put their file in your `eta-mlops` folder. Then:

```
uv run python imbalance.py --kappa labels_<yourname>.csv labels_<friend>.csv
```

You will see something like this (your numbers will differ):

```
agree on 15 of 20 orders
Cohen's kappa = 0.52    (1.0 = perfect, 0 = chance)
```

Think Kappa between 0.4 and 0.6 is "moderate agreement". You both looked carefully. Can a model learn a label better than the humans who made it?

Part 2 · 99% accurate and useless (20 min)

Target: `severely_late` — a delivery that took more than 90 minutes. Only about **1 in 100** orders.

```
uv run python imbalance.py
```

About 10–15 seconds. You should see (short version):

model	accuracy	precision	recall	caught	missed	false_alarms
plain	0.9913	0.691	0.15	65	369	29
class_weight=balanced	0.9477	0.08	0.429	186	248	2138

the plain model is 99.1% accurate and catches 65 of 434 severe delays.

Think 1. 99.1% accurate, and it misses **369 out of 434** late deliveries. Would this number look good in a presentation? 2. Class weights catch 186 instead of 65, but raise 2,138 false alarms. Who should decide if that trade is worth it?

Part 3 · The contest (20 min)

Metric: the best recall you can get while precision stays at 0.10 or more. **Score to beat:** 0.410.

First write `contest_score(y_true, proba)` in `imbalance.py`:

- try many thresholds `t` (for example 200 values between `proba.min()` and `proba.max()`)
- for each one, `pred = proba >= t`
- skip thresholds where precision is below `min_precision`
- return (best recall, the threshold that gave it)

```
uv run pytest tests/test_slot08.py
uv run python imbalance.py
```

The last line now shows `CONTEST SCORE`. To beat 0.410, change the model inside `main()`: more features, different settings, or resampling from `imblearn`.

If something goes wrong

what you see	what to do
kappa is <code>nan</code>	One of you used only 1s or only 0s. Look again at a few borderline orders.
<code>kappa</code> error about lengths	You and your friend must use the same 20 orders (made with <code>--make-labels</code>).
contest score is 0	Precision never reached 0.10. Use <code>predict_proba(...)[:, 1]</code> , and try lower thresholds.
accuracy goes up with class weights	You are predicting the wrong column. It must be <code>severely_late</code> .